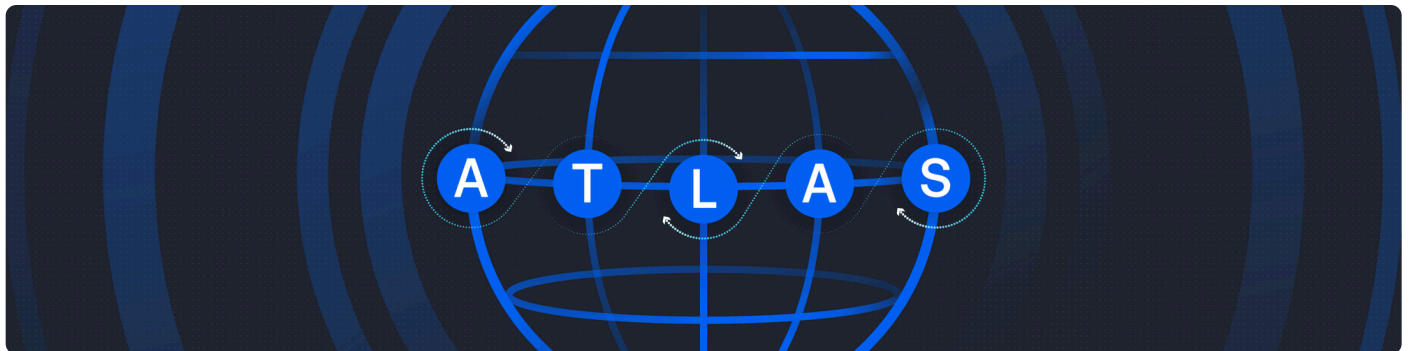


RESEARCH

AdapTive-LeArning Speculator System (ATLAS): A New Paradigm in LLM Inference via Runtime-Learning Accelerators

ATLAS delivers up to 4x faster LLM inference, powered by Together Turbo's latest research.

OCTOBER 10, 2025 · BY JUNXIONG WANG, SHIRLEY WU, ZELEI SHAO, VIKRANTH SRIVATSA, JUE WANG, ROY YUAN, QINGYANG WU, ALPAY ARIYAK, RUPERT WU, WAI TONG CHUNG, CHENFENG XU, YONATAN OREN, PRAGAASH PONNUSAMY, YINENG ZHANG, AVNER MAY, LEON SONG, TRI DAO, PERCY LIANG, CE ZHANG, BEN ATHIWARATKUN



At Together AI, the AI Native Cloud, we're obsessed with performance. Making large language models faster, cheaper, and more efficient is not a one-trick problem — it requires optimizing along multiple axes. That is the philosophy behind **Together Turbo**, our suite of inference innovations that draw from research in algorithms, architectures, and modeling recipes. We're excited to introduce the AdapTive-LeArning Speculator System (ATLAS), the first speculator of its kind that gives automatic performance improvements without any manual tuning.

ATLAS offers a new way of doing speculative decoding — one that dynamically improves at runtime — and it fits seamlessly alongside our other Turbo techniques like the proprietary [Together Turbo Speculator](#) or [Custom Speculators](#). But why create an adaptive-learning speculator system?



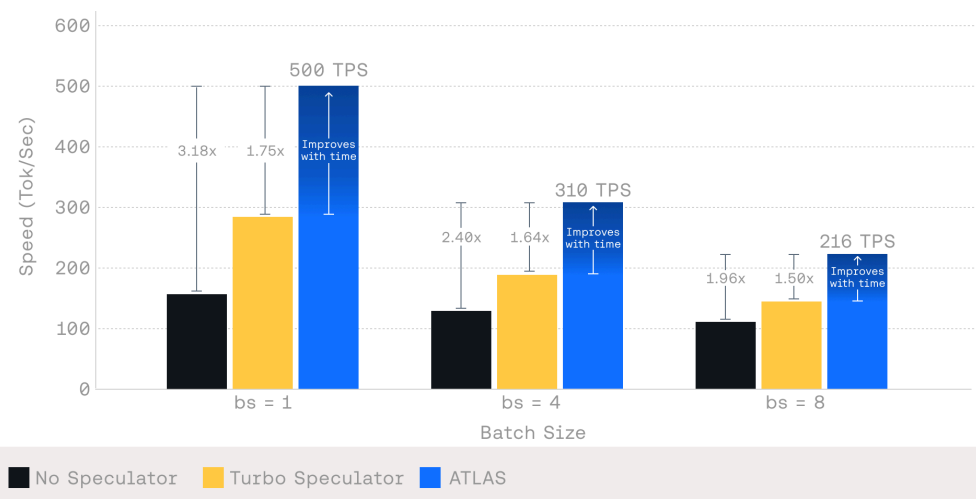
LINKS IN THIS ARTICLE

[Together Turbo Speculator](#)
[Custom Speculators](#)
[AI Research Careers](#)

Standard speculators are trained for general workloads. Custom speculators are trained on your specific data, but only for a specific snapshot in time. However, as the workload evolves (codebase grows, traffic patterns shift, request distributions change), even highly customized speculators can fall behind. In contrast, ATLAS evolves automatically with usage, learning from both historical patterns and live traffic to continuously align with the target model's behaviors in real time. This means the more you use our inference service, the better ATLAS will perform!

Built on top of Together Turbo Speculator, ATLAS reaches up to 500 TPS on DeepSeek-V3.1 and up to 460 TPS on Kimi-K2 in a fully adapted scenario — 2.65x faster than standard decoding, outperforming even specialized hardware like Groq (Figure 1).

Adaptive-Learning Speculator System (ATLAS) Performance On DeepSeek V3.1 (Arena Hard)



Adaptive-Learning Speculator System (ATLAS) Performance On Kimi-K2 0905 (Arena Hard)

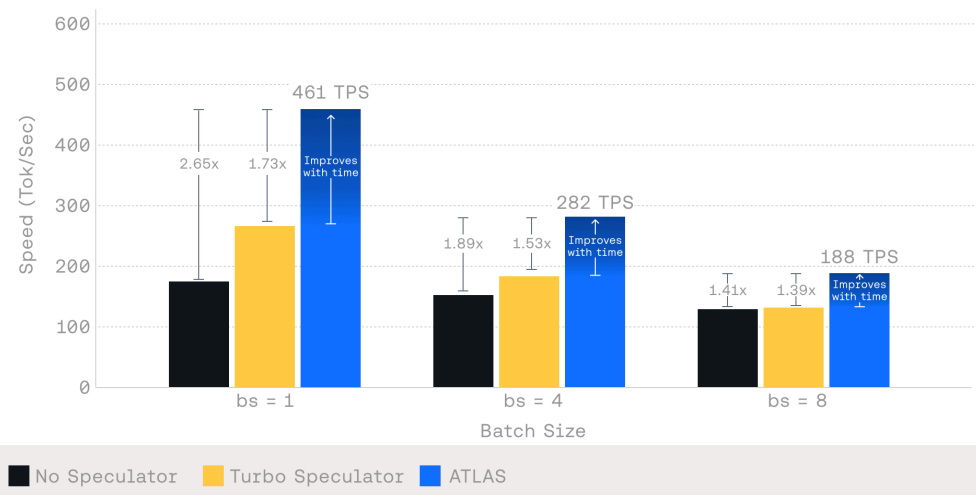


Figure 1: We show the decoding speed on NVIDIA HGX B200 with our Turbo speculator and the adaptive-learning speculator system for DeepSeek-V3.1 (top) KIMI-K2-0905 (bottom) with Arena Hard traffic.¹

1. Speculative Decoding

Speculative decoding is one of the most powerful levers for accelerating inference.² Instead of having the target model generate every token step by step, a faster

speculator (also known as the **draft model**) proposes multiple tokens ahead, and the target model *verifies* them in parallel in a single forward pass. The verification process ensures that the quality of the output matches the distribution of non-speculative decoding, while achieving speedups by accepting many tokens at a time.

The overall speed is influenced by the acceptance rate α (i.e., how often the target model agrees with the drafted tokens from the speculator) and the relative latency c of the draft versus the target. Typically, larger speculators with more parameters yield higher acceptance rates due to their higher capacity but are slower to generate draft tokens. Progress therefore comes from both sides: aligning draft and target models to increase α (training objectives, data, and algorithms) and designing draft models/kernels that keep c low while maintaining α (sparsity, quantization, lightweight & kernel-efficient architectures). The sweet spot is where a high α meets a low c , minimizing end-to-end latency.

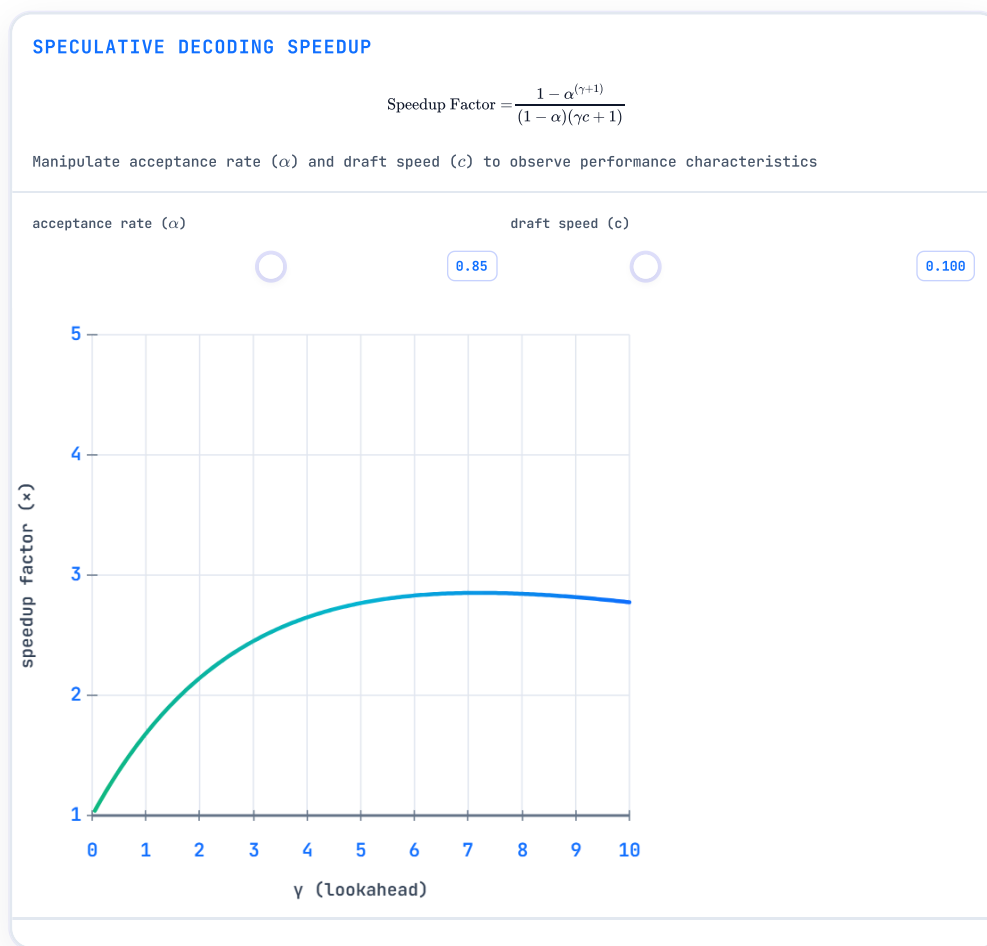


Figure 2: Interactive speedup calculator for speculative decoding. Different optimization techniques influence acceptance rate or draft latency, creating a complex optimization space to maximize speedup. The optimal lookahead (γ) itself varies with model configuration – strong speculators (high α , low c) continue gaining from longer lookaheads ($\gamma = 5+$), while weaker configurations (low α , high c) plateau early ($\gamma = 3-4$).

At Together AI, the Turbo team has developed high-performance speculators that have achieved the world's fastest decoding speeds on NVIDIA Blackwell by drawing on advances across architecture, sparsity, algorithms, post-training recipes, and data [1-9]. We've built a **speculator design and selection framework** that determines the

optimal speculator architecture (width/depth, lookahead, sparsity/quantization, KV reuse) and a **scalable training system** that brings up speculators for the largest and most challenging open-source targets quickly and reproducibly (e.g., DeepSeek-V3.1 and Kimi-K2). For instance, while Kimi ships without a ready-to-use speculator, we can train and deploy one rapidly and take Kimi from ~150 TPS out of the box to 270+ TPS on the same hardware and batch settings, while preserving target-model quality (see Figure 1, yellow bars). This pipeline powers Turbo Speculators that deliver state-of-the-art decoding latency, and it sets the stage for what comes next: **an Adaptive-Learning Speculator System** that adjusts token drafting to the workload in real time.

2. Introducing Turbo’s Adaptive-Learning Speculator System

At Together AI, we power a broad range of inference workloads. But today’s speculative decoding methods are constrained to using a **static** speculator, trained on a fixed dataset. Once deployed, the speculator cannot adapt, leading to degrading performance if the input distribution evolves. This problem is particularly pronounced in serverless, multi-tenant environments, where input diversity is sky-high. New users continuously arrive, and bring with them unique workloads that the fixed speculator may not have seen during training. Furthermore, these speculators typically use a *fixed lookahead*, predicting the same number of tokens regardless of the speculator’s confidence. Put simply, *a static speculator cannot keep up*.

Adaptive-Learning Speculator System Design

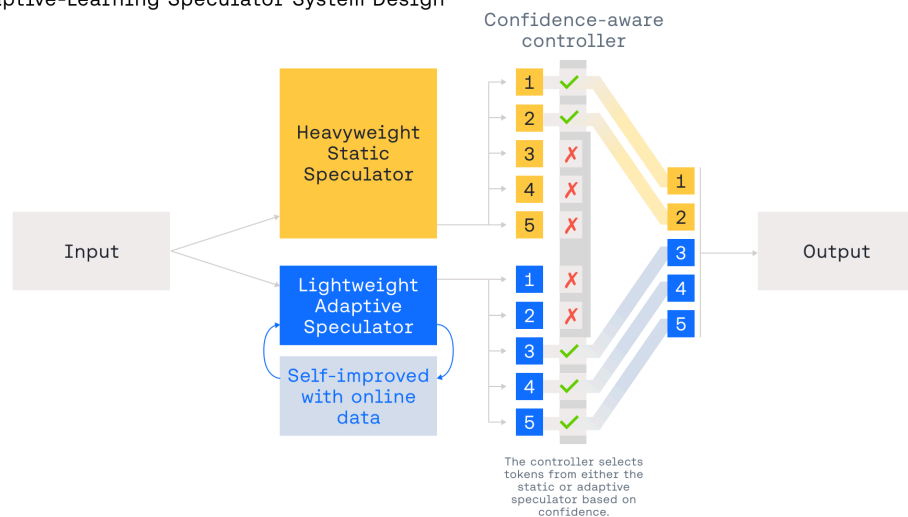


Figure 3: Two speculators—one static, one adaptive—work with a confidence-aware controller that selects between them and adjusts lookahead for optimal accuracy and speed.

To address these limitations, we designed the **Adaptive-Learning Speculative System** with two cooperating speculators, as shown in Figure 3:

- A **heavyweight static speculator** trained on a broad corpus that provides strong, general speculation.
- A **lightweight adaptive speculator** that allows for rapid, low-overhead updates from real-time traffic, specializing on-the-fly to emerging domains.

- A **confidence-aware controller** that chooses which speculator to trust at each step and what speculation lookahead to use, using longer speculations when the speculator has high confidence.

Efficiency Guardrail via Static Speculator. The static Turbo Speculator serves as an always-on speed floor: it is trained on a broad corpus and remains stable across workloads, so TPS does not collapse when traffic shifts or the adaptive path is cold. In ATLAS, we use it to jump-start speed and provide a fail-safe fallback—if confidence drops or drift is detected, the controller shortens lookahead or routes back to the static path to preserve latency while the adaptive speculator relearns.

Customized Speculator vs. Adaptive-Learning. We know from our previous studies that a customized speculator trained on samples from real traffic that mirror expected usage delivers an additional speed boost. The Adaptive-Learning Speculator enables us to be even more customized in real time. For instance, during a vibe-coding session, the adaptive system can specialize a lightweight speculator for the relevant code files being edited and not seen during training, further increasing the acceptance rate and decoding speed. This kind of on-the-fly specialization is hard to achieve with static speculators.

Accelerating RL Training. Reinforcement learning (RL) alternates between two phases: (1) a rollout phase, where the current policy generates trajectories and receives rewards, and (2) an update phase, where we use the rewards to update the policy. In practice, rollouts are often the bottleneck, accounting for roughly 70% of total wall-clock time³. In general, because the policy distribution shifts throughout training, static speculators quickly fall out of alignment with the target policy, resulting in sub-optimal throughput.⁴ ATLAS addresses this by adapting online to the evolving policy and the specific RL domain, maintaining alignment and reducing the overall rollout time. The domain-specific, iterative nature of RL further enables rapid adaptation, yielding sustained and growing speedups. As shown in Figure 4, applying ATLAS to the RL-MATH pipeline produces increasing speedups as training progresses.

Acceptance Rate Evolving During The RL Training

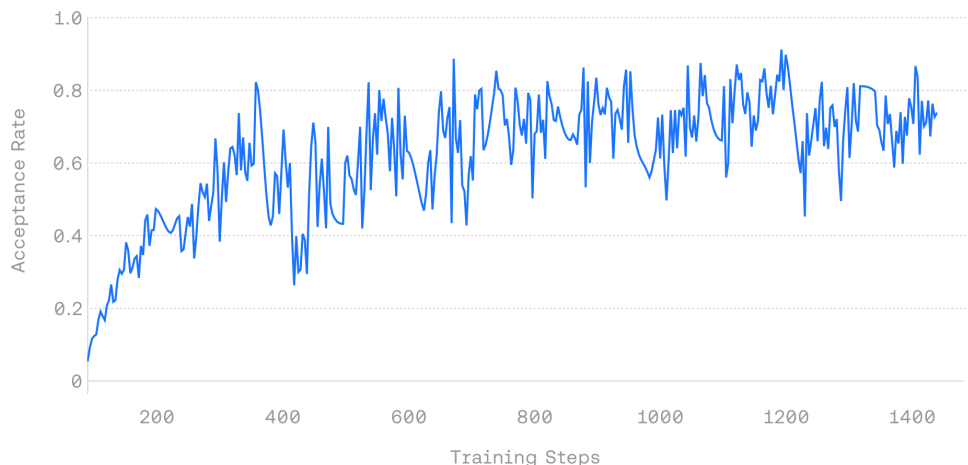


Figure 4: We train Qwen/Qwen2.5-7B-Instruct-1M on DeepScaler subsets using ATLAS for RL on NVIDIA Hopper H100 GPUs. The acceptance rate rises from below 10% to above 80% over 1.4k training steps, resulting in a more than 60% reduction for overall training time without changing RL training algorithm.

Built as part of the Turbo optimization suite. The Adaptive-Learning Speculator System is a core component of the broader Turbo optimization suite, where each layer of optimization compounds the benefits of the others. As illustrated in Figure 5, performance progressively improves through near-lossless quantization (calibrated to preserve quality), the Turbo Speculator, and finally the Adaptive-Learning Speculator System. Additional optimizations in the suite include TurboBoost-TTFT (not shown) for reducing time-to-first-token latency, further contributing to end-to-end acceleration.

The Together Turbo Optimization Suite

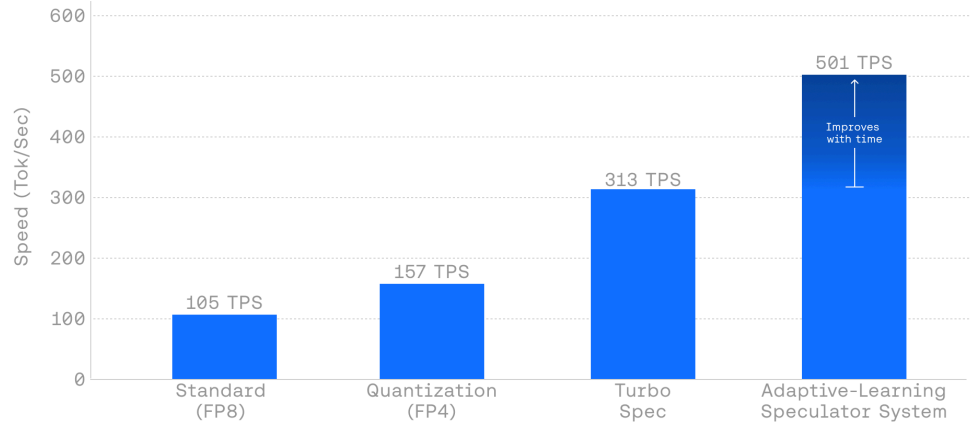


Figure 5: Performance gains across the Together Turbo Optimization Suite, showing progressive speed improvements from vanilla FP8 DeepSeek baseline (105 TPS) to the Adaptive-Learning Speculator System (501 TPS). The experiment is based on NVIDIA HGX B200 with batch-size 1, as detailed in Figure 1.

Extreme Peak Efficiency. When the input distribution is narrow and outputs closely echo previously seen tokens, the adaptive system specializes quickly. In this scenario, the controller becomes confident in utilizing more tokens from the lightweight speculator and lengthens lookahead tokens. This yields consistently higher TPS than static or one-off custom speculators can maintain. As shown in Figures 1 and 5, once fully adapted to Arena-Hard traffic, DeepSeek achieves up to 500 tokens per second for batch size 1 on 4 B200 GPUs, delivering a 400% speedup over the FP8 baseline (improvement from 105 TPS to 501 TPS).

Get ATLAS on your Dedicated Endpoint

Contact Us

Related Work

We build on a vibrant ecosystem of research in speculative decoding—from early *draft-and-verify* methods like [Fast Inference from Transformers via Speculative Decoding](#) and [Speculative Sampling](#), to structured variants including [Medusa](#), [OPT-Tree](#), and the [EAGLE](#) series. More recent studies made speculation adaptive: [Online](#)

[Speculative Decoding](#), [SpecDec++](#), [PEARL](#), [BanditSpec](#), and [HeteroSpec](#) tune speculation dynamically; and [OmniDraft](#), [Learning to Draft \(LTD\)](#), and [SpecRouter](#) explore cross-model adaptation, routing, and joint learning strategies. Related ideas like [SpecTr](#), [REST](#), and [Lookahead Decoding](#) further broaden the design space. ATLAS synthesizes these advances into a dual-speculator framework that balances stability and adaptability via a confidence-aware controller making real-time decisions.

Build the Future of Efficient AI

In parallel to making models smarter and more capable, advancements in inference efficiency are just as transformative – because intelligence only matters when you can deliver it swiftly, cost-effectively, and at scale. At Together AI, our Turbo team turns cutting-edge research (algorithms, quantization, sparsity, distillation, architectures, model pruning, and post-training recipes) into production systems that cut costs and unlock entirely new product experiences. If you love turning elegant ideas into billions of faster tokens, obsess over optimizing efficiency-quality frontier, and want your research to land in real users' hands quickly, come build with us. We're hiring exceptional research scientists and engineers who can push the frontier of efficient AI. [Apply](#) to Together and help define how intelligence is deployed.

Footnotes

1. Each benchmark burst contained 32 [arena-hard](#) prompts ($\approx 3,000$ token completions on average). Since the observed TPS depends on the prompts while speculative decoding is used, we report tokens per second (TPS) as the mean across requests. In the Adaptive-Learning Speculator System scenario, we show the peak speed where the system is fully adapted to Arena Hard traffic. We use TP=4 for DeepSeek-V3.1 and TP=8 for Kimi-K2-0905 on NVIDIA B200. This is different from our previous [blog](#) where we used TP=8, EP=2 for DeepSeek.
2. [Fast Inference from Transformers via Speculative Decoding](#).
3. [DeepCoder: A Fully Open-Source 14B Coder at O3-mini Level](#) [DeepSWE: Training a Fully Open-sourced, State-of-the-Art Coding Agent by Scaling RL](#)
4. Speculative decoding is typically not beneficial in pure throughput- or compute-bound settings, where GPUs are fully saturated. However, in reinforcement learning (RL) training, the situation can be different. Agent RL training often operates with small batch sizes and CPU-driven environment steps, where each agent waits for model outputs to do the next action. In this regime, endpoint latency (tokens per second per request) becomes the bottleneck. This makes speculative decoding highly applicable for RL training, if there is a suitable and high performant speculator. Faster decoding pipeline can improve CPU utilization and overall sample throughput.

References

- [1] Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads

[2] SpecExec: Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices

[3] Ladder-Residual: Parallelism-Aware Architecture for Accelerating Large Model Inference with Communication Overlapping

[4] TEAL: Training-Free Activation Sparsity in Large Language Model

[5] The Mamba in the Llama: Distilling and Accelerating Hybrid Models

[6] SEQUOIA: Scalable and Robust Speculative Decoding

[7] Mixture-of-Agents Alignment: Harnessing the Collective Intelligence of Open-Source LLMs to Improve Post-Training

[8] Boosting DeepSeek-R1's Speed with Customized Speculative Decoding

[9] DeepSWE: Training a Fully Open-sourced, State-of-the-Art Coding Agent by Scaling RL

Subscribe to newsletter

your@email.com



Products

Solutions

Research

Blog





- About
- Careers
- Pricing
- Contact
- Support
- Status
- Trust Center

- Consent Preferences
- Cookie Policy
- Privacy policy
- Terms of service

© 2025
San Francisco,
CA 94114